

Digital Design and Computer Architecture

RISC-V Edition



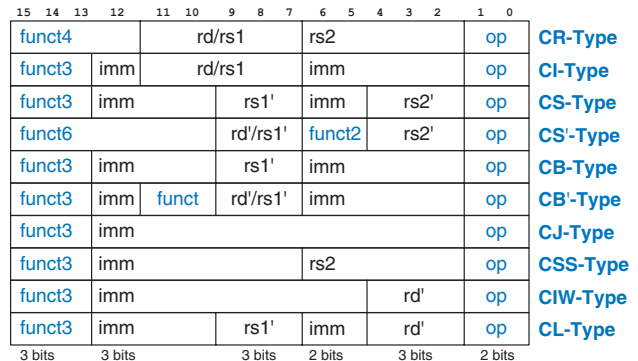
Digital Design and Computer Architecture

RISC-V Edition



Table B.4 Register names and numbers

Name	Register Number	Use
zero	x0	Constant value 0
ra	x1	Return address
sp	x2	Stack pointer
gp	x3	Global pointer
tp	x4	Thread pointer
t0–2	x5–7	Temporary registers
s0/fp	x8	Saved register / Frame pointer
s1	x9	Saved register
a0–1	x10–11	Function arguments / Return values
a2–7	x12–17	Function arguments
s2–11	x18–27	Saved registers
t3–6	x28–31	Temporary registers

**Figure B.2 RISC-V compressed (16-bit) instruction formats****Table B.5 RVM: RISC-V multiply and divide instructions**

op	funct3	funct7	Type	Instruction	Description	Operation
0110011 (51)	000	0000001	R	mul rd, rs1, rs2	multiply	$rd = (rs1 * rs2)_{31:0}$
0110011 (51)	001	0000001	R	mulh rd, rs1, rs2	multiply high signed signed	$rd = (rs1 * rs2)_{63:32}$
0110011 (51)	010	0000001	R	mulhsu rd, rs1, rs2	multiply high signed unsigned	$rd = (rs1 * rs2)_{63:32}$
0110011 (51)	011	0000001	R	mulhu rd, rs1, rs2	multiply high unsigned unsigned	$rd = (rs1 * rs2)_{63:32}$
0110011 (51)	100	0000001	R	div rd, rs1, rs2	divide (signed)	$rd = rs1 / rs2$
0110011 (51)	101	0000001	R	divu rd, rs1, rs2	divide unsigned	$rd = rs1 / rs2$
0110011 (51)	110	0000001	R	rem rd, rs1, rs2	remainder (signed)	$rd = rs1 \% rs2$
0110011 (51)	111	0000001	R	remu rd, rs1, rs2	remainder unsigned	$rd = rs1 \% rs2$

Table B.6 RVC: RISC-V compressed (16-bit) instructions

op	instr _{15:10}	funct2	Type	RVC Instruction	32-Bit Equivalent
00 (0)	000---	—	CIW	c.addi4spn rd', sp, imm	addi rd', sp, ZeroExt(imm)*4
00 (0)	001---	—	CL	c.fld fd', imm(rs1')	fld fd', (ZeroExt(imm)*8)(rs1')
00 (0)	010---	—	CL	c.lw rd', imm(rs1')	lw rd', (ZeroExt(imm)*4)(rs1')
00 (0)	011---	—	CL	c.flw fd', imm(rs1')	flw fd', (ZeroExt(imm)*4)(rs1')
00 (0)	101---	—	CS	c.fsd fs2', imm(rs1')	fsd fs2', (ZeroExt(imm)*8)(rs1')
00 (0)	110---	—	CS	c.sw rs2', imm(rs1')	sw rs2', (ZeroExt(imm)*4)(rs1')
00 (0)	111---	—	CS	c.fsw fs2', imm(rs1')	fsw fs2', (ZeroExt(imm)*4)(rs1')
01 (1)	000000	—	CI	c.nop (rs1=0,imm=0)	nop
01 (1)	000---	—	CI	c.addi rd, imm	addi rd, rd, SignExt(imm)
01 (1)	001---	—	CJ	c.jal label	jal ra, label
01 (1)	010---	—	CI	c.li rd, imm	addi rd, x0, SignExt(imm)
01 (1)	011---	—	CI	c.lui rd, imm	lui rd, {14{imm ₅ }, imm}
01 (1)	011---	—	CI	c.addi16sp sp, imm	addi sp, sp, SignExt(imm)*16
01 (1)	100--0	—	CB'	c.srli rd', imm	srli rd', rd', imm
01 (1)	100-01	—	CB'	c.srai rd', imm	srai rd', rd', imm
01 (1)	100-10	—	CB'	c.andi rd', imm	andi rd', rd', SignExt(imm)
01 (1)	100011	00	CS'	c.sub rd', rs2'	sub rd', rd', rs2'
01 (1)	100011	01	CS'	c.xor rd', rs2'	xor rd', rd', rs2'
01 (1)	100011	10	CS'	c.or rd', rs2'	or rd', rd', rs2'
01 (1)	100011	11	CS'	c.and rd', rs2'	and rd', rd', rs2'
01 (1)	101---	—	CJ	c.j label	jal x0, label
01 (1)	110---	—	CB	c.beqz rs1', label	beq rs1', x0, label
01 (1)	111---	—	CB	c.bnez rs1', label	bne rs1', x0, label
10 (2)	000---	—	CI	c.slli rd, imm	slli rd, rd, imm
10 (2)	001---	—	CI	c.fldsp fd, imm	fld fd, (ZeroExt(imm)*8)(sp)
10 (2)	010---	—	CI	c.lwsp rd, imm	lw rd, (ZeroExt(imm)*4)(sp)
10 (2)	011---	—	CI	c.flwsp fd, imm	flw fd, (ZeroExt(imm)*4)(sp)
10 (2)	1000--	—	CR	c.jr rs1 (rs1≠0,rs2=0)	jalr x0, rs1, 0
10 (2)	1000--	—	CR	c.mv rd, rs2 (rd≠0,rs2≠0)	add rd, x0, rs2
10 (2)	1001--	—	CR	c.ebreak (rs1=0,rs2=0)	ebreak
10 (2)	1001--	—	CR	c.jalr rs1 (rs1≠0,rs2=0)	jalr ra, rs1, 0
10 (2)	1001--	—	CR	c.add rd, rs2 (rs1≠0,rs2≠0)	add rd, rd, rs2
10 (2)	101---	—	CSS	c.fsdsp fs2, imm	fsd fs2, (ZeroExt(imm)*8)(sp)
10 (2)	110---	—	CSS	c.swsp rs2, imm	sw rs2, (ZeroExt(imm)*4)(sp)
10 (2)	111---	—	CSS	c.fswsp fs2, imm	fsw fs2, (ZeroExt(imm)*4)(sp)

rs1', rs2', rd': 3-bit register designator for registers 8–15: 000₂ = x8 or f8, 001₂ = x9 or f9, etc.

表 B.4 寄存器名称和编号 Na

我	注册 数字使用	
zero	x0	Constant value 0
ra	x1	Return address
sp	x2	Stack pointer
gp	x3	Global pointer
tp	x4	Thread pointer
t0-2	x5-7	Temporary registers
s0/fp	x8	Saved register / Frame pointer
s1	x9	Saved register
a0-1	x10-11	Function arguments / Return value 是的
a2-7	x12-17	Function arguments
s2-11	x18-27	Saved registers
t3-6	x28-31	Temporary registers

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

func4 rd/rs1 rs2 op CR-类型								
func3	imm	rd/rs1		imm		op		CI-T类型
func3	imm		rs1'	imm	rs2'	op		CS-T类型
func6			rd'/rs1'	func2	rs2'	op		CS'-类型
func3	imm		rs1'	imm		op		CB-T类型
func3	imm	func2	rd'/rs1'	imm		op		CB'-类型
func3	imm						op	CJ-T类型
func3	imm			rs2		op		CSS 类型
func3	imm					rd'	op	CIW 类型
func3	imm		rs1'	imm	rd'	op		CL-T类型
3 bits	3 bits		3 bits	2 bits	3 bits	2 bits		

Table B.7 RISC-V pseudoinstructions

Pseudoinstruction	RISC-V Instructions	Description	Operation
nop	addi x0, x0, 0	no operation	
li rd, imm _{11:0}	addi rd, x0, imm _{11:0}	load 12-bit immediate	rd = SignExtend(imm _{11:0})
li rd, imm _{31:0}	lui rd, imm _{31:12} addi rd, rd, imm _{11:0}	load 32-bit immediate	rd = imm _{31:0}
mv rd, rs1	addi rd, rs1, 0	move (also called “register copy”)	rd = rs1
not rd, rs1	xori rd, rs1, -1	one’s complement	rd = ~rs1
neg rd, rs1	sub rd, x0, rs1	two’s complement	rd = -rs1
seqz rd, rs1	sltiu rd, rs1, 1	set if = 0	rd = (rs1 == 0)
snez rd, rs1	sltu rd, x0, rs1	set if ≠ 0	rd = (rs1 ≠ 0)
sltz rd, rs1	slt rd, rs1, x0	set if < 0	rd = (rs1 < 0)
sgtz rd, rs1	slt rd, x0, rs1	set if > 0	rd = (rs1 > 0)
beqz rs1, label	beq rs1, x0, label	branch if = 0	if (rs1 == 0) PC = label
bnez rs1, label	bne rs1, x0, label	branch if ≠ 0	if (rs1 ≠ 0) PC = label
blez rs1, label	bge x0, rs1, label	branch if ≤ 0	if (rs1 ≤ 0) PC = label
bgez rs1, label	bge rs1, x0, label	branch if ≥ 0	if (rs1 ≥ 0) PC = label
bltz rs1, label	blt rs1, x0, label	branch if < 0	if (rs1 < 0) PC = label
bgtz rs1, label	blt x0, rs1, label	branch if > 0	if (rs1 > 0) PC = label
ble rs1, rs2, label	bge rs2, rs1, label	branch if ≤	if (rs1 ≤ rs2) PC = label
bgt rs1, rs2, label	blt rs2, rs1, label	branch if >	if (rs1 > rs2) PC = label
bleu rs1, rs2, label	bgeu rs2, rs1, label	branch if ≤ (unsigned)	if (rs1 ≤ rs2) PC = label
bgtu rs1, rs2, label	bltu rs2, rs1, offset	branch if > (unsigned)	if (rs1 > rs2) PC = label
j label	jal x0, label	jump	PC = label
jal label	jal ra, label	jump and link	PC = label, ra = PC + 4
jr rs1	jalr x0, rs1, 0	jump register	PC = rs1
jalr rs1	jalr ra, rs1, 0	jump and link register	PC = rs1, ra = PC + 4
ret	jalr x0, ra, 0	return from function	PC = ra
call label	jal ra, label	call nearby function	PC = label, ra = PC + 4
call label	auipc ra, offset _{31:12} jalr ra, ra, offset _{11:0}	call far away function	PC = PC + offset, ra = PC + 4
la rd, symbol	auipc rd, symbol _{31:12} addi rd, rd, symbol _{11:0}	load address of global variable	rd = PC + symbol
l{b h w} rd, symbol	auipc rd, symbol _{31:12} l{b h w} rd, symbol _{11:0} (rd)	load global variable	rd = [PC + symbol]
s{b h w} rs2, symbol, rs1	auipc rs1, symbol _{31:12} s{b h w} rs2, symbol _{11:0} (rs1)	store global variable	[PC + symbol] = rs2
crr rd, csr	crrs rd, csr, x0	read CSR	rd = csr
crrw csr, rs1	crrw x0, csr, rs1	write CSR	csr = rs1

* If bit 11 of the immediate / offset / symbol is 1, the upper immediate is incremented by 1. symbol and offset are the 32-bit PC-relative addresses of a label and a global variable, respectively.

Table B.8 Privileged / CSR instructions

op	funct3	Type	Instruction	Description	Operation
1110011 (115)	000	I	ecall	transfer control to OS (imm=0)	
1110011 (115)	000	I	ebreak	transfer control to debugger (imm=1)	
1110011 (115)	000	I	uret	return from user exception (rs1=0,rd=0,imm=2)	PC = uepc
1110011 (115)	000	I	sret	return from supervisor exception (rs1=0,rd=0,imm=258)	PC = sepc
1110011 (115)	000	I	mret	return from machine exception (rs1=0,rd=0,imm=770)	PC = mepc
1110011 (115)	001	I	crrw rd,csr,rs1	CSR read/write (imm=CSR number)	rd = csr, csr = rs1
1110011 (115)	010	I	crrs rd,csr,rs1	CSR read/set (imm=CSR number)	rd = csr, csr = csr rs1
1110011 (115)	011	I	crrc rd,csr,rs1	CSR read/clear (imm=CSR number)	rd = csr, csr = csr & ~rs1
1110011 (115)	101	I	crrwi rd,csr,uimm	CSR read/write immediate (imm=CSR number)	rd = csr, csr = ZeroExt(uimm)
1110011 (115)	110	I	crrsi rd,csr,uimm	CSR read/set immediate (imm=CSR number)	rd = csr, csr = csr ZeroExt(uimm)
1110011 (115)	111	I	crrci rd,csr,uimm	CSR read/clear immediate (imm=CSR number)	rd = csr, csr = csr & ~ZeroExt(uimm)

For privileged / CSR instructions, the 5-bit unsigned immediate, uimm, is encoded in the rs1 field.

表 B.7 RISC-V 伪指令 伪指令 RISC-V 指令 描述 操作 nop add

i x0, x			
	零	no operation	
li rd, imm _{11:0}	addi rd, x0, imm _{11:0}	load 12-bit immediate	rd = SignExtend(imm _{11:0})
li rd, imm _{31:0}	lui rd, imm _{31:12} addi rd, rd, imm _{11:0}	load 32-bit immediate	rd = imm _{31:0}
mv rd, rs1	addi rd, rs1, 0	move (also called “register copy”)	rd = rs1
not rd, rs1	xori rd, rs1, -1	one’s complement	rd = ~rs1
neg rd, rs1	sub rd, x0, rs1	two’s complement	rd = -rs1
seqz rd, rs1	sltiu rd, rs1, 1	set if = 0	rd = (rs1 == 0)
snez rd, rs1	sltu rd, x0, rs1	set if ≠ 0	rd = (rs1 ≠ 0)
sltz rd, rs1	slt rd, rs1, x0	set if < 0	rd = (rs1 < 0)
sgtz rd, rs1	slt rd, x0, rs1	set if > 0	rd = (rs1 > 0)
beqz rs1, label	beq rs1, x0, label	branch if = 0	if (rs1 == 0) PC = label
bnez rs1, label	bne rs1, x0, label	branch if ≠ 0	if (rs1 ≠ 0) PC = label
blez rs1, label	bge x0, rs1, label	branch if ≤ 0	if (rs1 ≤ 0) PC = label
bgez rs1, label	bge rs1, x0, label	branch if ≥ 0	if (rs1 ≥ 0) PC = label
bltz rs1, label	blt rs1, x0, label	branch if < 0	if (rs1 < 0) PC = label
bgtz rs1, label	blt x0, rs1, label	branch if > 0	if (rs1 > 0) PC = label
ble rs1, rs2, label	bge rs2, rs1, label	branch if ≤	if (rs1 ≤ rs2) PC = label
bgt rs1, rs2, label	blt rs2, rs1, label	branch if >	if (rs1 > rs2) PC = label
bleu rs1, rs2, label	bgeu rs2, rs1, label	branch if ≤ (unsigned)	if (rs1 ≤ rs2) PC = label
bgtu rs1, rs2, label	bltu rs2, rs1, offset	branch if > (unsigned)	if (rs1 > rs2) PC = label
j label	jal x0, label	jump	PC = label
jal label	jal ra, label	jump and link	PC = label, ra = PC + 四
jr rs1	jalr x0, rs1, 0	jump register	PC = rs1
jalr rs1	jalr ra, rs1, 0	jump and link register	PC = rs1, ra = PC + 四
ret	jalr x0, ra, 0	return from function	PC = ra
call label	jal ra, label	call nearby function	PC = label, ra = PC + 四
call label	auipc ra, offset _{31:12} jalr ra, ra, offset _{11:0}	call far away function	PC = PC + offset, ra = PC + 四
la rd, symbol	auipc rd, symbol _{31:12} addi rd, rd, symbol _{11:0}	load address of global variable	rd = PC + symbol
l{b h w} rd, symbol	auipc rd, symbol _{31:12} l{b h w} rd, symbol _{11:0} (rd)	load global variable	rd = [PC + symbol]
s{b h w} rs2, symbol, rs1	auipc rs1, symbol _{31:12} s{b h w} rs2, symbol _{11:0} (rs1)	store global variable	[PC + symbol] = rs2
crr rd, csr	crrs rd, csr, x0	read CSR	rd = csr
crrw csr, rs1	crrw x0, csr, rs1	write CSR	csr = rs1

* If bit 11 of the immediate / offset / symbol is 1, the upper immediate is incremented by 1. symbol and offset are the 32-bit PC-relative addresses of a 亚贝尔 and a global variable, respectively.

表 B.8 特权 / CSR 指令

操作码	funct3	类型	指令	描述	操作	
1110011 (115)	000		我呼叫		将控制权转移到操作系统 (imm=0)	
1110011 (115)	000	I	ebreak		transfer control to debugger (imm=1)	
1110011 (115)	000	I	uret		return from user exception (rs1=0,rd=0,imm=2)	PC = uepc
1110011 (115)	000	I	sret		return from supervisor exception (rs1=0,rd=0,imm=258)	PC = sepc
1110011 (115)	000	I	mret		return from machine exception (rs1=0,rd=0,imm=770)	PC = mepc
1110011 (115)	001	I	crrw rd,csr,rs1		CSR read/write (imm=CSR number)	rd = csr, csr = rs1
1110011 (115)	010	I	crrs rd,csr,rs1		CSR read/set (imm=CSR number)	rd = csr, csr = csr rs1
1110011 (115)	011	I	crrc rd,csr,rs1		CSR read/clear (imm=CSR number)	rd = csr, csr = csr & ~rs1
1110011 (115)	101	I	crrwi rd,csr,uimm		CSR read/write immediate (imm=CSR number)	rd = csr, csr = ZeroExt(u免疫
1110011 (115)	110	I	crrsi rd,csr,uimm		CSR read/set immediate (imm=CSR number)	rd = csr, csr = csr ZeroExt(uim米
1110011 (115)	111	I	crrci rd,csr,uimm		CSR read/clear immediate (imm=CSR number)	rd = csr, csr = csr & ~ZeroExt(uim米

For privileged / CSR instructions, the 5-bit unsigned immediate, uimm, is encoded in the rs1 field.

In Praise of *Digital Design and Computer Architecture*

RISC-V Edition

Harris and Harris have shown how to design a RISC-V processor from the gates all the way up through the microarchitecture. Their clear explanations combined with their comprehensive approach give a full picture of both digital design and the RISC-V architecture. With the exciting opportunity that students have to run large digital designs on modern FPGAs, this approach is both informative and enlightening.

David A. Patterson University of California, Berkeley

What broad fields Harris and Harris have distilled into one book! As semiconductor manufacturing matures, the importance of digital design and computer architecture will only increase. Readers will find an approachable and comprehensive treatment of both topics and will walk away with a clear understanding of the RISC-V instruction set architecture.

Andrew Waterman SiFive

There are excellent texts for teaching digital design and there are excellent texts for teaching computer hardware organization - and this textbook does both! It is also unique in its ability to connect the dots. The writing makes the RISC-V architecture understandable by building on the basics, and I have found the exercises to be a great resource across multiple courses.

Roy Kravitz Portland State University

When I first read Harris and Harris's MIPS textbook back in 2008, I thought that it was one of the best books I had ever read for teaching computer architecture. I started using it in my courses immediately. Thirteen years later, I have had the honor of reviewing this new RISC-V edition, and my opinion of their books has not changed: *Digital Design and Computer Architecture: RISC-V Edition* is an excellent book, very clear, thorough, with a high educational value, and in line with the courses that we teach in the areas of digital design and computer architecture. I look forward to using this RISC-V textbook in my courses.

Daniel Chaver Martinez University Complutense of Madrid

赞美 *Digital Design and Computer Architecture*

RISC-V 版本

Harris 和 Harris 展示了如何从门电路一直设计到微架构来设计一个 RISC-V 处理器。他们清晰的解释结合全面的方法，提供了数字设计和 RISC-V 架构的完整图景。对于学生有机会在现代 FPGA 上运行大型数字设计来说，这种方法既有信息性又具有启发性。

戴维·A 帕特森 加州大学伯克利分校

哈里斯和哈里斯将如此广泛的领域精炼成了一本书！随着半导体制造的成熟，数字设计和计算机体系结构的重要性只会增加。读者将会发现对这两个主题的处理既易于理解又全面，并且能够清楚地理解 RISC-V 指令集架构。

安德鲁·沃特曼 SiFive

有极好的教材用于教授数字设计，也有极好的教材用于教授计算机硬件组织——而这本教科书两者都涵盖！它还独特地能够将各个知识点联系起来。通过在基础知识上构建内容，写作使 RISC-V 架构易于理解，我发现这些练习在多个课程中都是极好的资源。

罗伊·克拉维茨 波特兰州立大学

当我在2008年第一次阅读Harris和Harris的MIPS教材时，我认为这是我读过的关于计算机体系结构教学中最好的书之一。我立即开始在我的课程中使用它。十三年后，我有幸审阅这本新的RISC-V版，我对他们书籍的看法没有改变： *Digital Design*

and Computer Architecture: RISC-V Edition 是一本优秀的书，非常清晰、详尽，具有很高的教育价值，并且符合我们在数字设计和计算机体系结构领域教授的课程。我期待在我的课程中使用这本RISC-V教材。

Daniel Chaver Martinez University Complutense of Madrid

Digital Design and Computer Architecture

RISC-V Edition

数字设计与计算机体系结构

RISC-V 版本

Digital Design and Computer Architecture

RISC-V Edition

Sarah L. Harris
David Harris



ELSEVIER

AMSTERDAM • BOSTON • HEIDELBERG • LONDON
NEW YORK • OXFORD • PARIS • SAN DIEGO
SAN FRANCISCO • SINGAPORE • SYDNEY • TOKYO

Morgan Kaufmann is an imprint of Elsevier

MK
MORGAN KAUFMANN

数字设计与计算机体系 结构

RISC-V 版本

莎拉·L·哈里斯
大卫·哈里斯



ELSEVIER

AMSTERDAM • BOSTON • HEIDELBERG • LONDON
NEW YORK • OXFORD • PARIS • SAN DIEGO
SAN FRANCISCO • SINGAPORE • SYDNEY • TOKYO

Morgan Kaufmann is an imprint of Elsevier

MK
MORGAN KAUFMANN

Morgan Kaufmann is an imprint of Elsevier
50 Hampshire Street, 5th Floor, Cambridge, MA 02139, United States

Copyright © 2022 Elsevier Inc. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or any information storage and retrieval system, without permission in writing from the publisher. Details on how to seek permission, further information about the Publisher's permissions policies and our arrangements with organizations such as the Copyright Clearance Center and the Copyright Licensing Agency, can be found at our website: www.elsevier.com/permissions.

This book and the individual contributions contained in it are protected under copyright by the Publisher (other than as may be noted herein).

Notices

Knowledge and best practice in this field are constantly changing. As new research and experience broaden our understanding, changes in research methods, professional practices, or medical treatment may become necessary.

Practitioners and researchers must always rely on their own experience and knowledge in evaluating and using any information, methods, compounds, or experiments described herein. In using such information or methods they should be mindful of their own safety and the safety of others, including parties for whom they have a professional responsibility.

To the fullest extent of the law, neither the Publisher nor the authors, contributors, or editors, assume any liability for any injury and/or damage to persons or property as a matter of products liability, negligence or otherwise, or from any use or operation of any methods, products, instructions, or ideas contained in the material herein.

British Library Cataloguing-in-Publication Data

A catalogue record for this book is available from the British Library.

Library of Congress Cataloging-in-Publication Data

A catalog record for this book is available from the Library of Congress.

ISBN: 978-0-12-820064-3

For Information on all Morgan Kaufmann publications visit our website at
<https://www.elsevier.com/books-and-journals>

Publisher: Katey Birtcher

Senior Acquisitions Editor: Steve Merken

Editorial Project Manager: Ruby Gammell/Andrae Akeh

Senior Project Manager: Manikandan Chandrasekaran

Cover Designer: Victoria Pearson Esser

Typeset by MPS Limited, Chennai, India

Printed in the United States of America

Last digit is the print number: 9 8 7 6 5 4 3 2 1



Working together
to grow libraries in
developing countries

www.elsevier.com • www.bookaid.org

摩根考夫曼是爱思唯尔的一个品牌，地址：美国马萨诸塞州剑桥市汉普郡街50号，5楼，邮编02139

版权 © 2022 爱思唯尔公司版权所有。

未经出版商书面许可，本出版物的任何部分不得以任何形式或任何手段复制或传输，包括电子或机械方式，如复印、录音或任何信息存储与检索系统。关于如何申请许可、了解出版商许可政策的更多信息以及我们与版权清算中心（Copyright Clearance Center）和版权许可机构（Copyright Licensing Agency）等组织的合作安排，可在我们的网站上找到：www.elsevier.com/permissions。

本书及其中包含的个人贡献受出版商的版权保护（除非本文另有说明）。

通知

这一领域的知识和最佳实践在不断变化。随着新的研究和经验拓宽我们的理解，研究方法、专业实践或医疗治疗可能需要发生变化。

从业者和研究人员在评估和使用本文中描述的任何信息、方法、化合物或实验时，必须始终依靠自己的经验和知识。在使用这些信息或方法时，他们应注意自身安全以及他人的安全，包括他们负有专业责任的相关方。

在法律允许的最大范围内，无论是出版商还是作者、贡献者或编辑，对于因产品责任、疏忽或其他原因造成的人员或财产的任何伤害和/或损害，或因本材料中包含的任何方法、产品、说明或理念的使用或操作所引起的任何损害，不承担任何责任。

英国图书馆出版物编目数据

英国图书馆提供了这本书的目录记录。

国会图书馆出版物编目数据 本书的编目记录可从国会图书馆获取。

ISBN: 978-0-12-820064-3

For Information on all Morgan Kaufmann publications visit our website at <https://www.elsevier.com/books-and-journals>

Publisher: 凯蒂·伯彻 *Senior Acquisitions Editor:* 史蒂夫·默肯 *Editorial Project Manager:* 露比·加梅尔/安德烈·阿凯 *Senior Project Manager:* 马尼康丹·钱德拉塞卡兰 *Cover Designer:* 维多利亚·皮尔森·埃瑟

Typeset by MPS Limited, Chennai, India

Printed in the United States of America

最后一位数字是印刷编号：9 8 7 6 5 4 3 2 1



Working together
to grow libraries in
developing countries

www.elsevier.com • www.bookaid.org

Contents

Preface	xix
Features	xx
Online Supplements	xxi
How to Use the Software Tools in a Course.....	xxii
Labs	xxiii
RVfpga.....	xxiii
Bugs	xxiv
Acknowledgments.....	xxiv
About the Authors	xxv
Chapter 1 From Zero to One	1
1.1 The Game Plan	1
1.2 The Art of Managing Complexity.....	2
1.2.1 <i>Abstraction</i>	2
1.2.2 <i>Discipline</i>	3
1.2.3 <i>The Three -Y's</i>	4
1.3 The Digital Abstraction.....	5
1.4 Number Systems	7
1.4.1 <i>Decimal Numbers</i>	7
1.4.2 <i>Binary Numbers</i>	7
1.4.3 <i>Hexadecimal Numbers</i>	9
1.4.4 <i>Bytes, Nibbles, and All That Jazz</i>	11
1.4.5 <i>Binary Addition</i>	12
1.4.6 <i>Signed Binary Numbers</i>	13
1.5 Logic Gates.....	17
1.5.1 <i>NOT Gate</i>	18
1.5.2 <i>Buffer</i>	18
1.5.3 <i>AND Gate</i>	18
1.5.4 <i>OR Gate</i>	19
1.5.5 <i>Other Two-Input Gates</i>	19
1.5.6 <i>Multiple-Input Gates</i>	19
1.6 Beneath the Digital Abstraction.....	20
1.6.1 <i>Supply Voltage</i>	20
1.6.2 <i>Logic Levels</i>	20
1.6.3 <i>Noise Margins</i>	21

目录

前言	xix
特征	x
x 在线补充材料	xxi
如何在课程中使用软件工具	xxii
实验	xxiii
RVfpga	xxiii
错误	xxiv
致谢	xxiv
关于作者	xxv
第1章 从零到一	1
1.1 游戏计划	1
1.2 管理复杂性的艺术	2
1.2.1 <i>Abstraction</i>	2
1.2.2 <i>Discipline</i>	3
1.2.3 <i>The Three -Y's</i>	4
1.3 数字抽象	5
1.4 数字系统	7
1.4.1 <i>Decimal Numbers</i>	7
1.4.2 <i>Binary Numbers</i>	7
1.4.3 <i>Hexadecimal Numbers</i>	7
1.4.4 <i>Bytes, Nibbles, and All That Jazz</i>	9
1.5 逻辑门	17
1.5.1 <i>NOT Gate</i>	18
1.5.2 <i>Buffer</i>	18
1.5.3 <i>AND Gate</i>	18
1.5.4 <i>OR Gate</i>	19
1.5.5 <i>Other Two-Input Gates</i>	19
1.5.6 <i>Multiple-Input Gates</i>	19
1.6 数字抽象的背后	20
1.6.1 <i>Supply Voltage</i>	20
1.6.2 <i>Logic Levels</i>	20
1.6.3 <i>Noise Margins</i>	20
21	21

1.6.4	DC Transfer Characteristics	22
1.6.5	The Static Discipline	22
1.7	CMOS Transistors	24
1.7.1	Semiconductors	25
1.7.2	Diodes	25
1.7.3	Capacitors	26
1.7.4	nMOS and pMOS Transistors	26
1.7.5	CMOS NOT Gate.....	29
1.7.6	Other CMOS Logic Gates	29
1.7.7	Transmission Gates.....	31
1.7.8	Pseudo-nMOS Logic.....	31
1.8	Power Consumption.....	32
1.9	Summary and a Look Ahead	34
	Exercises	36
	Interview Questions	50
Chapter 2 Combinational Logic Design		53
2.1	Introduction	53
2.2	Boolean Equations	56
2.2.1	Terminology	56
2.2.2	Sum-of-Products Form.....	56
2.2.3	Product-of-Sums Form.....	58
2.3	Boolean Algebra	58
2.3.1	Axioms.....	59
2.3.2	Theorems of One Variable	59
2.3.3	Theorems of Several Variables	60
2.3.4	The Truth Behind It All	62
2.3.5	Simplifying Equations	63
2.4	From Logic to Gates.....	64
2.5	Multilevel Combinational Logic.....	67
2.5.1	Hardware Reduction	68
2.5.2	Bubble Pushing	69
2.6	X's and Z's, Oh My	71
2.6.1	Illegal Value: X.....	71
2.6.2	Floating Value: Z.....	72
2.7	Karnaugh Maps.....	73
2.7.1	Circular Thinking	74
2.7.2	Logic Minimization with K-Maps.....	75
2.7.3	Don't Cares.....	79
2.7.4	The Big Picture.....	80
2.8	Combinational Building Blocks.....	81
2.8.1	Multiplexers	81
2.8.2	Decoders	84

1.6.4 DC Transfer Characteristics	22
1.6.5 The Static Discipline	22 1.7 C
MOS 晶体管	24 1.7.1
Semiconductors	25 1.7.2
Diodes	25 1.7.3
Capacitors	26 1.7.4
nMOS and pMOS Transistors	26 1.7.5
CMOS NOT Gate	29 1.7.6
Other CMOS Logic Gates	29 1.7.7
Transmission Gates	31 1.7.8
Pseudo-nMOS Logic.	31 1.8 功耗 .
.....	
章节 r 2 组合逻辑设计	 53
2.1 引言	53 2.2 布
尔方程	56 2.2.1
Terminology	56 2.2.2
Sum-of-Products Form	56 2.2.3
Product-of-Sums Form	58 2.3 布尔代
数	58 2.3.1 Axioms ..
.....	59 2.3.2
Theorems of One Variable	59 2.3.3
Theorems of Several Variables	60 2.3.4
The Truth Behind It All	62 2.3.5
Simplifying Equations	63 2.4 从逻辑
到门电路	

2.9	Timing	86
2.9.1	<i>Propagation and Contamination Delay</i>	86
2.9.2	<i>Glitches</i>	90
2.10	Summary	93
	Exercises	95
	Interview Questions	104
Chapter 3 Sequential Logic Design		107
3.1	Introduction	107
3.2	Latches and Flip-Flops	107
3.2.1	<i>SR Latch</i>	109
3.2.2	<i>D Latch</i>	111
3.2.3	<i>D Flip-Flop</i>	112
3.2.4	<i>Register</i>	112
3.2.5	<i>Enabled Flip-Flop</i>	113
3.2.6	<i>Resettable Flip-Flop</i>	114
3.2.7	<i>Transistor-Level Latch and Flip-Flop Designs</i>	114
3.2.8	<i>Putting It All Together</i>	116
3.3	Synchronous Logic Design	117
3.3.1	<i>Some Problematic Circuits</i>	117
3.3.2	<i>Synchronous Sequential Circuits</i>	118
3.3.3	<i>Synchronous and Asynchronous Circuits</i>	120
3.4	Finite State Machines	121
3.4.1	<i>FSM Design Example</i>	121
3.4.2	<i>State Encodings</i>	127
3.4.3	<i>Moore and Mealy Machines</i>	130
3.4.4	<i>Factoring State Machines</i>	132
3.4.5	<i>Deriving an FSM from a Schematic</i>	135
3.4.6	<i>FSM Review</i>	138
3.5	Timing of Sequential Logic	139
3.5.1	<i>The Dynamic Discipline</i>	140
3.5.2	<i>System Timing</i>	140
3.5.3	<i>Clock Skew</i>	146
3.5.4	<i>Metastability</i>	149
3.5.5	<i>Synchronizers</i>	150
3.5.6	<i>Derivation of Resolution Time</i>	152
3.6	Parallelism	155
3.7	Summary	159
	Exercises	160
	Interview Questions	169

2.9 时间	86
9.1 <i>Propagation and Contamination Delay</i>	86
<i>Glitches</i>	90
.....	93
.....	95
.....	104
章 第3章 顺序逻辑设计	107
3.1 引言	107
寄存器与触发器	107
<i>SR Latch</i>	109
<i>D Latch</i>	111
<i>D Flip-Flop</i>	112
<i>Register</i>	112
<i>Enabled Flip-Flop</i>	113
<i>Resettable Flip-Flop</i>	114
<i>Transistor-Level Latch and Flip-Flop Designs</i>	114
.....	116
3.3 同步逻辑设计	
3.4.1 <i>FSM Design Example</i>	121
3.4.2 <i>State Encodings</i>	127
3.4.3 <i>Moore and Mealy Machines</i>	130
4.4 <i>Factoring State Machines</i>	132
4.5 <i>Deriving an FSM from a Schematic</i>	135
6 <i>FSM Review</i>	138
3.5 时序逻辑的时序	139
<i>The Dynamic Discipline</i>	140
<i>System Timing</i>	140
<i>Clock Skew</i>	146
<i>Metastability</i>	149
<i>Synchronizers</i>	150
<i>Derivation of Resolution Time</i>	152
性	155
.....	159
.....	160
.....	

Chapter 4	Hardware Description Languages	171
4.1	Introduction	171
4.1.1	Modules	171
4.1.2	Language Origins	172
4.1.3	Simulation and Synthesis	173
4.2	Combinational Logic	175
4.2.1	Bitwise Operators	175
4.2.2	Comments and White Space	178
4.2.3	Reduction Operators	178
4.2.4	Conditional Assignment	179
4.2.5	Internal Variables	180
4.2.6	Precedence	182
4.2.7	Numbers	183
4.2.8	Z's and X's	184
4.2.9	Bit Swizzling	186
4.2.10	Delays	186
4.3	Structural Modeling	188
4.4	Sequential Logic	191
4.4.1	Registers	191
4.4.2	Resettable Registers	192
4.4.3	Enabled Registers	194
4.4.4	Multiple Registers	195
4.4.5	Latches	196
4.5	More Combinational Logic	196
4.5.1	Case Statements	199
4.5.2	If Statements	200
4.5.3	Truth Tables with Don't Cares	203
4.5.4	Blocking and Nonblocking Assignments	203
4.6	Finite State Machines	207
4.7	Data Types	211
4.7.1	SystemVerilog	212
4.7.2	VHDL	213
4.8	Parameterized Modules	215
4.9	Testbenches	218
4.10	Summary	222
	Exercises	224
	Interview Questions	235
Chapter 5	Digital Building Blocks	237
5.1	Introduction	237
5.2	Arithmetic Circuits	237
5.2.1	Addition	237
5.2.2	Subtraction	244

第4章 硬件描述语言	171
4.1 引言	171 4.1.1
<i>Modules</i>	171 4.1.2
<i>Language Origins</i>	172 4.1.3
<i>Simulation and Synthesis</i>	173 4.2 组合逻辑
175 4.2.1	
<i>Bitwise Operators</i>	175 4.2.2
<i>Comments and White Space</i>	178 4.2.3
<i>Reduction Operators</i>	178 4.2.4
<i>Conditional Assignment</i>	179 4.2.5
<i>Internal Variables</i>	180 4.2.6
<i>Precedence</i>	182 4.2.7 <i>Numbers</i>
.....	
4.4.1 <i>Registers</i>	191
4.4.2 <i>Resettable Registers</i>	192 4
4.3 <i>Enabled Registers</i>	194 4.
4.4 <i>Multiple Registers</i>	195 4.
4.5 <i>Latches</i>	196
4.5 更多组合逻辑	196
4.5.1 <i>Case Statements</i>	199
4.5.2 <i>If Statements</i>	200
4.5.3 <i>Truth Tables with Don't Cares</i>	203 4.
4.5.4 <i>Blocking and Nonblocking Assignments</i>	203
4.6 有限状态机	207 4.7 数据类型
.....	211 4.7.1
<i>SystemVerilog</i>	212 4.7.2 <i>VHDL</i>
.....	213 4.8 参数化模块
.....	215 4.9 测试平台
.....	218 4.10 总结
.....	222 练习题
.....	224 面试问题
.....	235
第五章 数字构建模块	237
5.1 引言	237 5.2 算术电路
.....	237 5.2.1 <i>Addition</i>
.....	237 5.2.2 <i>Subtraction</i>
.....	244

5.2.3	<i>Comparators</i>	245
5.2.4	<i>ALU</i>	247
5.2.5	<i>Shifters and Rotators</i>	251
5.2.6	<i>Multiplication</i>	253
5.2.7	<i>Division</i>	254
5.2.8	<i>Further Reading</i>	255
5.3	<i>Number Systems</i>	256
5.3.1	<i>Fixed-Point Number Systems</i>	256
5.3.2	<i>Floating-Point Number Systems</i>	257
5.4	<i>Sequential Building Blocks</i>	261
5.4.1	<i>Counters</i>	261
5.4.2	<i>Shift Registers</i>	262
5.5	<i>Memory Arrays</i>	265
5.5.1	<i>Overview</i>	265
5.5.2	<i>Dynamic Random Access Memory (DRAM)</i>	267
5.5.3	<i>Static Random Access Memory (SRAM)</i>	268
5.5.4	<i>Area and Delay</i>	268
5.5.5	<i>Register Files</i>	269
5.5.6	<i>Read Only Memory (ROM)</i>	269
5.5.7	<i>Logic Using Memory Arrays</i>	271
5.5.8	<i>Memory HDL</i>	272
5.6	<i>Logic Arrays</i>	272
5.6.1	<i>Programmable Logic Array (PLA)</i>	275
5.6.2	<i>Field Programmable Gate Array (FPGA)</i>	276
5.6.3	<i>Array Implementations</i>	282
5.7	<i>Summary</i>	283
	<i>Exercises</i>	285
	<i>Interview Questions</i>	297
Chapter 6 Architecture		299
6.1	<i>Introduction</i>	299
6.2	<i>Assembly Language</i>	300
6.2.1	<i>Instructions</i>	301
6.2.2	<i>Operands: Registers, Memory, and Constants</i>	302
6.3	<i>Programming</i>	308
6.3.1	<i>Program Flow</i>	308
6.3.2	<i>Logical, Shift, and Multiply Instructions</i>	308
6.3.3	<i>Branching</i>	311
6.3.4	<i>Conditional Statements</i>	313
6.3.5	<i>Getting Loopy</i>	315
6.3.6	<i>Arrays</i>	317
6.3.7	<i>Function Calls</i>	320
6.3.8	<i>Pseudoinstructions</i>	330

5.2.3 Comparators	245
5.2.4 ALU	247
Shifters and Rotators	251
Multiplication	253
Division	254
Further Reading	255
统	256
Fixed-Point Number Systems	256
Floating-Point Number Systems	257
模块	261
.....	261
5.5.1 Overview	265
5.5.2 Dynamic Random Access Memory (DRAM)	267
5.3 Static Random Access Memory (SRAM)	268
4 Area and Delay	268
5 Register Files	269
6 Read Only Memory (ROM)	269
Logic Using Memory Arrays	271
Memory HDL	272
5.6 逻辑阵列	272
5.6.1 Programmable Logic Array (PLA)	275
6.2 Field Programmable Gate Array (FPGA)	276
3 Array Implementations	282
5.7 总结	283
练习	285
.....	297
第6章 建筑	299
6.1 引言	299
语言	300
.....	301
Operands: Registers, Memory, and Constants	302
.....	308
6.3.1 Program Flow	308
6.3.2 Logical, Shift, and Multiply Instructions	308
3.3 Branching	311
.3.4 Conditional Statements	313
3.5 Getting Loopy	315
3.6 Arrays	317
.3.7 Function Calls	320
.3.8 Pseudoinstructions	330

6.4	Machine Language	332
6.4.1	<i>R-Type Instructions</i>	332
6.4.2	<i>I-Type Instructions</i>	334
6.4.3	<i>S/B-Type Instructions</i>	336
6.4.4	<i>U/J-Type Instructions</i>	338
6.4.5	<i>Immediate Encodings</i>	340
6.4.6	<i>Addressing Modes</i>	341
6.4.7	<i>Interpreting Machine Language Code</i>	342
6.4.8	<i>The Power of the Stored Program</i>	343
6.5	Lights, Camera, Action: Compiling, Assembling, and Loading	344
6.5.1	<i>The Memory Map</i>	344
6.5.2	<i>Assembler Directives</i>	346
6.5.3	<i>Compiling</i>	348
6.5.4	<i>Assembling</i>	350
6.5.5	<i>Linking</i>	353
6.5.6	<i>Loading</i>	355
6.6	Odds and Ends	355
6.6.1	<i>Endianness</i>	355
6.6.2	<i>Exceptions</i>	356
6.6.3	<i>Signed and Unsigned Instructions</i>	360
6.6.4	<i>Floating-Point Instructions</i>	361
6.6.5	<i>Compressed Instructions</i>	362
6.7	Evolution of the RISC-V Architecture	363
6.7.1	<i>RISC-V Base Instruction Sets and Extensions</i>	364
6.7.2	<i>Comparison of RISC-V and MIPS Architectures</i>	365
6.7.3	<i>Comparison of RISC-V and ARM Architectures</i>	365
6.8	Another Perspective: x86 Architecture	366
6.8.1	<i>x86 Registers</i>	366
6.8.2	<i>x86 Operands</i>	367
6.8.3	<i>Status Flags</i>	369
6.8.4	<i>x86 Instructions</i>	369
6.8.5	<i>x86 Instruction Encoding</i>	371
6.8.6	<i>Other x86 Peculiarities</i>	372
6.8.7	<i>The Big Picture</i>	373
6.9	Summary	374
	Exercises	375
	Interview Questions	390
	Chapter 7 Microarchitecture	393
7.1	Introduction	393
7.1.1	<i>Architectural State and Instruction Set</i>	393
7.1.2	<i>Design Process</i>	394
7.1.3	<i>Microarchitectures</i>	396

6.4 机器语言	332
6.4.1 <i>R-Type Instructions</i>	332
6.4.2 <i>I-Type Instructions</i>	334
6.4.3 <i>S/B-Type Instructions</i>	336
6.4.4 <i>U/J-Type Instructions</i>	338
6.4.5 <i>Immediate Encodings</i>	340
6.4.6 <i>Addressing Modes</i>	341
6.4.7 <i>Interpreting Machine Language Code</i>	342
6.4.8 <i>The Power of the Stored Program</i>	343
6.5 灯光、摄像、行动：编译、组装和加载	
..... 344 6.5.1 <i>The Memory Map</i>	
..... 344 6.5.2 <i>Assembler Directives</i>	
... 346 6.5.3 <i>Compiling</i>	348
6.5.4 <i>Assembling</i>	350
6.5.5 <i>Linking</i>	353
6.5.6 <i>Loading</i>	355
6.6 杂项	355
6.6.1 <i>Endianness</i>	355
6.6.2 <i>Exceptions</i>	356
6.6.3 <i>Signed and Unsigned Instructions</i>	360
6.6.4 <i>Floating-Point Instructions</i>	361
6.6.5 <i>Compressed Instructions</i>	362
6.7 RISC-V 架构的发展	363
6.7.1 <i>RISC-V Base Instruction Sets and Extensions</i>	364
6.7.2 <i>Comparison of RISC-V and MIPS Architectures</i>	365
6.7.3 <i>Comparison of RISC-V and ARM Architectures</i>	365
6.8 另一种视角：x86架构	366
6.8.1 <i>x86 Registers</i>	366
6.8.2 <i>x86 Operands</i>	367
6.8.3 <i>Status Flags</i>	36
9 6.8.4 <i>x86 Instructions</i>	36
9 6.8.5 <i>x86 Instruction Encoding</i>	371
6.8.6 <i>Other x86 Peculiarities</i>	372
6.8.7 <i>The Big Picture</i>	373
6.9 摘要	374
..... 375 面试问题	
..... 390	
第7章 微架构	393
7.1 引言	393
7.1.1 <i>Architectural State and Instruction Set</i>	393
7.1.2 <i>Design Process</i>	394
7.1.3 <i>Microarchitectures</i>	396

7.2	Performance Analysis	397
7.3	Single-Cycle Processor	398
7.3.1	<i>Sample Program</i>	399
7.3.2	<i>Single-Cycle Datapath</i>	399
7.3.3	<i>Single-Cycle Control</i>	407
7.3.4	<i>More Instructions</i>	410
7.3.5	<i>Performance Analysis</i>	412
7.4	Multicycle Processor	415
7.4.1	<i>Multicycle Datapath</i>	416
7.4.2	<i>Multicycle Control</i>	422
7.4.3	<i>More Instructions</i>	432
7.4.4	<i>Performance Analysis</i>	435
7.5	Pipelined Processor	439
7.5.1	<i>Pipelined Datapath</i>	441
7.5.2	<i>Pipelined Control</i>	443
7.5.3	<i>Hazards</i>	443
7.5.4	<i>Performance Analysis</i>	454
7.6	HDL Representation	456
7.6.1	<i>Single-Cycle Processor</i>	457
7.6.2	<i>Generic Building Blocks</i>	461
7.6.3	<i>Testbench</i>	464
7.7	Advanced Microarchitecture	468
7.7.1	<i>Deep Pipelines</i>	468
7.7.2	<i>Micro-Operations</i>	469
7.7.3	<i>Branch Prediction</i>	470
7.7.4	<i>Superscalar Processors</i>	472
7.7.5	<i>Out-of-Order Processor</i>	473
7.7.6	<i>Register Renaming</i>	476
7.7.7	<i>Multithreading</i>	478
7.7.8	<i>Multiprocessors</i>	479
7.8	Real-World Perspective: Evolution of RISC-V Microarchitecture	482
7.9	Summary	486
	Exercises	488
	Interview Questions	497
Chapter 8 Memory Systems		499
8.1	Introduction	499
8.2	Memory System Performance Analysis	503
8.3	Caches	505
8.3.1	<i>What Data is Held in the Cache?</i>	505
8.3.2	<i>How is Data Found?</i>	506
8.3.3	<i>What Data is Replaced?</i>	514
8.3.4	<i>Advanced Cache Design</i>	515

7.2 性能分析	397
7.3 单周期处理器	398
7.3.1 Sample Program	399
7.3.2 Single-Cycle Datapath	399
7.3.3 Single-Cycle Control	407
7.3.4 More Instructions	410
7.3.5 Performance Analysis	412
7.4 多周期处理器	415
7.4.1 Multicycle Datapath	416
7.4.2 Multicycle Control	422
7.4.3 More Instructions	432
7.4.4 Performance Analysis	435
7.5 流水线处理器	439
7.5.1 Pipelined Datapath	441
7.5.2 Pipelined Control	443
7.5.3 Hazards	443
7.5.4 Performance Analysis	454
7.6 HDL 表示	456
7.6.1 Single-Cycle Processor	457
7.6.2 Generic Building Blocks	461
7.6.3 Testbench	464
7.7 高级微架构	468
7.7.1 Deep Pipelines	468
7.7.2 Micro-Operations	469
7.7.3 Branch Prediction	470
7.7.4 Superscalar Processors	472
7.7.5 Out-of-Order Processor	473
7.7.6 Register Renaming	476
7.7.7 Multithreading	478
7.7.8 Multiprocessors	479
7.8 现实世界视角: RISC-V 微架构的发展	482
7.9 总结	486
习题	488
面试题	497
第8章 记忆系统	499
8.1 引言	499
8.2 存储系统性能分析	503
8.3 缓存	505
8.3.1 What Data is Held in the Cache?	505
8.3.2 How is Data Found?	506
8.3.3 What Data is Replaced?	514
8.4 Advanced Cache Design	515

8.4	Virtual Memory.....	519
8.4.1	Address Translation.....	522
8.4.2	The Page Table.....	523
8.4.3	The Translation Lookaside Buffer.....	525
8.4.4	Memory Protection.....	526
8.4.5	Replacement Policies.....	527
8.4.6	Multilevel Page Tables	527
8.5	Summary.....	530
	Epilogue.....	530
	Exercises	532
	Interview Questions	541
	Chapter 9 Embedded I/O Systems	542
9.1	Introduction	542
	Chapter 9 is available as an online supplement	542.e1
9.1	Introduction	542.e1
9.2	Memory-Mapped I/O.....	542.e1
9.3	Embedded I/O Systems	542.e3
9.3.1	RED-V Board.....	542.e3
9.3.2	FE310-G002 System-on-Chip	542.e5
9.3.3	General-Purpose Digital I/O	542.e5
9.3.4	Device Drivers	542.e10
9.3.5	Serial I/O.....	542.e14
9.3.6	Timers.....	542.e29
9.3.7	Analog I/O	542.e30
9.3.8	Interrupts.....	542.e39
9.4	Other Microcontroller Peripherals	542.e43
9.4.1	Character LCDs.....	542.e43
9.4.2	VGA Monitor.....	542.e45
9.4.3	Bluetooth Wireless Communication	542.e53
9.4.4	Motor Control	542.e54
9.5	Summary.....	542.e64
	Appendix A Digital System Implementation	543
A.1	Introduction	543
	Appendix A is available as an online supplement	543.e1
A.1	Introduction	543.e1
A.2	74xx Logic	543.e1
A.2.1	Logic Gates.....	543.e2
A.2.2	Other Functions.....	543.e2

A.3	Programmable Logic	543.e2
A.3.1	PROMs	543.e2
A.3.2	PLAs	543.e6
A.3.3	FPGAs	543.e7
A.4	Application-Specific Integrated Circuits	543.e9
A.5	Datasheets	543.e9
A.6	Logic Families	543.e15
A.7	Switches and Light-Emitting Diodes	543.e17
A.7.1	Switches	543.e17
A.7.2	LEDs	543.e18
A.8	Packaging and Assembly	543.e19
A.8.1	Packages	543.e19
A.8.2	Breadboards	543.e20
A.8.3	Printed Circuit Boards	543.e20
A.8.4	Putting It All Together	543.e23
A.9	Transmission Lines	543.e23
A.9.1	Matched Termination	543.e24
A.9.2	Open Termination	543.e26
A.9.3	Short Termination	543.e27
A.9.4	Mismatched Termination	543.e27
A.9.5	When to Use Transmission Line Models	543.e30
A.9.6	Proper Transmission Line Terminations	543.e30
A.9.7	Derivation of Z_0	543.e32
A.9.8	Derivation of the Reflection Coefficient	543.e33
A.9.9	Putting It All Together	543.e34
A.10	Economics	543.e35

Appendix B RISC-V Instruction Set Summary 544

Appendix C C Programming 545

C.1	Introduction	545
-----	--------------------	-----

Appendix C is available as an online supplement 545.e1

C.1	Introduction	545.e1
C.2	Welcome to C	545.e3
C.2.1	C Program Dissection	545.e3
C.2.2	Running a C Program	545.e4
C.3	Compilation	545.e5
C.3.1	Comments	545.e5
C.3.2	#define	545.e5
C.3.3	#include	545.e6

A.3 可编程逻辑	543.e2 A.3.1
PROMs	543.e2 A.3.2 PLAs ..
.....	543.e6 A.3.3 FPGAs
.....	543.e7 A.4 专用集成电路
.....	543.e9 A.5 数据表
.....	543.e9 A.6 逻辑家族
.....	543.e15 A.7 开关和发光二极管
43.e17 A.7.1 <i>Switches</i>	543.e17
A.7.2 <i>LEDs</i>	543.e18 A.8 封装
A.9.1 <i>Matched Termination</i>	543.e24 A
.9.2 <i>Open Termination</i>	543.e26 A.
9.3 <i>Short Termination</i>	543.e27 A.9
.4 <i>Mismatched Termination</i>	543.e27 A.9.5
<i>When to Use Transmission Line Models</i>	543.e30 A.9.6
<i>Proper Transmission Line Terminations</i>	543.e30 A.9.7
<i>Derivation of Z_0</i>	543.e32 A.9.8
<i>Derivation of the Reflection Coefficient</i>	543.e33 A.9.9
<i>Putting It All Together</i>	543.e34
A.10 经济学	543.e35
附录 B RISC-V 指令集摘要	544
附录 C C 编程	545
C.1 引言	545
附录C可作为在线补充材料获取	545.e1
C.1 引言	545.e1C.2 欢
迎来到 C	545.e3C.2.1
<i>C Program Dissection</i>	545.e3C.2.2
<i>Running a C Program</i>	545.e4C.3 编译 ...
.....	545.e5C.3.1 <i>Comments</i>
.....	545.e5C.3.2 <i>#define</i>
.....	545.e5C.3.3 <i>#include</i>
.....	545.e6

C.4	Variables	545.e7
C.4.1	<i>Primitive Data Types</i>	545.e8
C.4.2	<i>Global and Local Variables</i>	545.e9
C.4.3	<i>Initializing Variables</i>	545.e11
C.5	Operators	545.e11
C.6	Function Calls	545.e15
C.7	Control-Flow Statements	545.e16
C.7.1	<i>Conditional Statements</i>	545.e17
C.7.2	<i>Loops</i>	545.e19
C.8	More Data Types.....	545.e21
C.8.1	<i>Pointers</i>	545.e21
C.8.2	<i>Arrays</i>	545.e23
C.8.3	<i>Characters</i>	545.e27
C.8.4	<i>Strings</i>	545.e27
C.8.5	<i>Structures</i>	545.e29
C.8.6	<i>typedef</i>	545.e31
C.8.7	<i>Dynamic Memory Allocation</i>	545.e32
C.8.8	<i>Linked Lists</i>	545.e33
C.9	Standard Libraries.....	545.e35
C.9.1	<i>stdio</i>	545.e35
C.9.2	<i>stdlib</i>	545.e40
C.9.3	<i>math</i>	545.e42
C.9.4	<i>string</i>	545.e42
C.10	Compiler and Command Line Options.....	545.e43
C.10.1	<i>Compiling Multiple C Source Files</i>	545.e43
C.10.2	<i>Compiler Options</i>	545.e43
C.10.3	<i>Command Line Arguments</i>	545.e44
C.11	Common Mistakes	545.e44
	Further Reading	547
	Index	549

C.4 变量	545.e7	C.4.1
<i>Primitive Data Types</i>	545.e8	C.4.2
<i>Global and Local Variables</i>	545.e9	C.4.3
<i>Initializing Variables</i>	545.e11	C.5 运算符
.....	545.e11	C.6 函数调用 ..
.....	545.e15	C.7 控制流语句
.....	545.e16	C.7.1 <i>Conditional Statements</i>
.....	545.e17	C.7.2 <i>Loops</i>
.....	545.e19	C.8 更多数据类型
545.e21		
C.8.1 <i>Pointers</i>	545.e21	C.
8.2 <i>Arrays</i>	545.e23	
C.8.3 <i>Characters</i>	545.e27	
C.8.4 <i>Strings</i>	545.e27	
C.8.5 <i>Structures</i>	545.e2	
9 C.8.6 <i>typedef</i>	545.e	
31 C.8.7 <i>Dynamic Memory Allocation</i>	545.e32	
C.8.8 <i>Linked Lists</i>	545.e33	
C.9 标准库	545.e35	
C.9.1 <i>stdio</i>	545.e35	
C.9.2 <i>stdlib</i>	545.e40	
C.9.3 <i>math</i>	545.e42	C
.9.4 <i>string</i>	545.e42	
C.10 编译器和命令行选项	545.e43	
C.10.1 <i>Compiling Multiple C Source Files</i>	545.e43	
C.10.2 <i>Compiler Options</i>	545.e43	
C.10.3 <i>Command Line Arguments</i>	545.e44	
C.11 常见错误	545.e44	
进一步阅读	547	
索引	549	

Preface

This book is unique in its treatment in that it presents digital logic design from the perspective of computer architecture, starting at the beginning with 1's and 0's and leading through to the design of a microprocessor.

We believe that building a microprocessor is a special rite of passage for engineering and computer science students. The inner workings of a processor seem almost magical to the uninitiated yet prove to be straightforward when carefully explained. Digital design in and of itself is a powerful and exciting subject. Assembly language programming unveils the inner language spoken by the processor. Microarchitecture is the link that brings it all together.

The first two versions of this increasingly popular text cover the MIPS and ARM architectures. As one of the original Reduced Instruction Set Computing architectures, MIPS is clean and exceptionally easy to understand and build. MIPS remains an important architecture, as it has inspired many of the subsequent architectures, including RISC-V. The ARM architecture has exploded in popularity over the past several decades because of its efficiency and rich ecosystem. More than 50 billion ARM processors have been shipped, and more than 75% of humans on the planet use products with ARM processors.

Over the past decade, RISC-V has emerged as an increasingly important architecture, both pedagogically and commercially. As the first widely used open-source computer architecture, RISC-V offers the simplicity of MIPS with the flexibility and features of modern processors.

Pedagogically, the learning objectives of the MIPS, ARM, and RISC-V editions are identical. The RISC-V architecture has a number of features, including extendibility and compressed instructions, that contribute to its efficiency but add a small amount of complexity. The three microarchitectures are also similar, with MIPS and RISC-V architectures sharing many similarities. We expect to offer MIPS, ARM, and RISC-V editions as long as the market demands.

前言

这本书在其处理方式上是独特的，因为它从计算机体系结构的角度介绍数字逻辑设计，从一开始的1和0开始，一直讲到微处理器的设计。

我们相信，构建微处理器对于工程和计算机科学的学生来说是一种特殊的成年礼。对于外行人来说，处理器的内部工作几乎显得神奇，但经过仔细讲解后，它证明其实是很直接的。数字设计本身就是一个强大且令人兴奋的学科。汇编语言编程揭示了处理器所使用的内部语言。微架构是将这一切联系在一起的纽带。

这本日益受欢迎的教材的前两个版本涵盖了MIPS和ARM架构。作为最初的精简指令集计算（RISC）架构之一，MIPS设计简洁，非常容易理解和构建。MIPS仍然是一种重要的架构，因为它启发了许多后续的架构，包括RISC-V。ARM架构在过去几十年里因其高效性和丰富的生态系统而爆发性增长。已经出货的ARM处理器超过500亿，全球超过75%的人使用搭载ARM处理器的产品。

在过去的十年中，RISC-V已经成为一种越来越重要的架构，无论是在教学上还是商业上。作为第一个广泛使用的开源计算机架构，RISC-V提供了MIPS的简洁性以及现代处理器的灵活性和功能。

从教学角度来看，MIPS、ARM和RISC-V版本的学习目标是相同的。RISC-V架构具有许多特性，包括可扩展性和压缩指令，这些特性提高了其效率，但增加了一点复杂性。这三种微架构也很相似，MIPS和RISC-V架构有许多相似之处。我们预计只要市场有需求，就会提供MIPS、ARM和RISC-V版本。

FEATURES

Side-by-Side Coverage of SystemVerilog and VHDL

Hardware description languages (HDLs) are at the center of modern digital design practices. Unfortunately, designers are evenly split between the two dominant languages, SystemVerilog and VHDL. This book introduces HDLs in [Chapter 4](#) as soon as combinational and sequential logic design has been covered. HDLs are then used in [Chapters 5](#) and [7](#) to design larger building blocks and entire processors. Nevertheless, [Chapter 4](#) can be skipped and the later chapters are still accessible for courses that choose not to cover HDLs.

This book is unique in its side-by-side presentation of SystemVerilog and VHDL, enabling the reader to learn the two languages. [Chapter 4](#) describes principles that apply to both HDLs, and then provides language-specific syntax and examples in adjacent columns. This side-by-side treatment makes it easy for an instructor to choose either HDL and for the reader to transition from one to the other, either in a class or in professional practice.

RISC-V Architecture and Microarchitecture

[Chapters 6](#) and [7](#) offer in-depth coverage of the RISC-V architecture and microarchitecture. RISC-V is an ideal architecture because it is a real architecture shipped in an increasing number of commercial products, yet it is streamlined and easy to learn. Moreover, because of its popularity in the commercial and hobbyist worlds, simulation and development tools exist for the RISC-V architecture.

Real-World Perspectives

In addition to the real-world perspective in discussing the RISC-V architecture, [Chapter 6](#) illustrates the architecture of Intel x86 processors to offer another perspective. [Chapter 9](#) (available as an online supplement) also describes peripherals in the context of SparkFun's RED-V RedBoard, a popular development board that centers on SiFive's Freedom E310 RISC-V processor. These real-world perspective chapters show how the concepts in the chapters relate to the chips found in many PCs and consumer electronics.

Accessible Overview of Advanced Microarchitecture

[Chapter 7](#) includes an overview of modern high-performance microarchitectural features, including branch prediction, superscalar, and out-of-order operation, multithreading, and multicore processors. The

特点

SystemVerilog 和 VHDL 的并排覆盖

硬件描述语言（HDL）是现代数字设计实践的核心。不幸的是，设计师在两种主要语言——SystemVerilog 和 VHDL 之间几乎平分秋色。本书在第4章介绍 HDL，时间是在组合逻辑和顺序逻辑设计讲解之后。然后，第5章和第7章将 HDL 用于设计更大的构建模块和完整的处理器。然而，第4章可以跳过，对于选择不讲解 HDL 的课程，后面的章节仍然可以使用。

本书在 SystemVerilog 和 VHDL 的并排呈现方面独具特色，使读者能够学习这两种语言。第 4 章描述了适用于两种 HDL 的原理，然后在相邻的列中提供语言特定的语法和示例。这种并排呈现方式使教师能够轻松选择任意一种 HDL，同时也便于读者在课堂或专业实践中从一种语言过渡到另一种语言。

RISC-V 架构与微架构

第6章和第7章深入介绍了RISC-V架构和微体系结构。RISC-V是一种理想的架构，因为它是一种真正的架构，已经应用在越来越多的商业产品中，同时它又精简且易于学习。此外，由于其在商业和爱好者领域的流行，RISC-V架构已经存在模拟和开发工具。

现实世界的观点

除了在讨论 RISC-V 架构时采用的现实世界视角外，第 6 章还介绍了 Intel x86 处理器的架构，以提供另一种视角。第 9 章（可作为在线附录获取）也在 SparkFun 的 RED-V RedBoard 上下文中描述了外设，这是一款以 SiFive 的 Freedom E310 RISC-V 处理器为中心的流行开发板。这些现实世界视角的章节展示了各章节中的概念如何与许多 PC 和消费电子产品中的芯片相关联。

先进微架构的易懂概述

第7章概述了现代高性能微架构特性，包括分支预测、超标量、乱序执行、多线程和多核处理器。

treatment is accessible to a student in a first course and shows how the microarchitectures in the book can be extended to modern processors.

End-of-Chapter Exercises and Interview Questions

The best way to learn digital design is to do it. Each chapter ends with numerous exercises to practice the material. The exercises are followed by a set of interview questions that our industrial colleagues have asked students who are applying for work in the field. These questions provide a helpful glimpse into the types of problems that job applicants will typically encounter during the interview process. Exercise solutions are available via the book's companion and instructor websites.

ONLINE SUPPLEMENTS

Supplementary materials are available online at ddcabook.com or the publisher's website: <https://www.elsevier.com/books-and-journals/book-companion/9780128200643>. These companion sites (accessible to all readers) include the following:

- ▶ Links to video lectures
- ▶ Solutions to odd-numbered exercises
- ▶ Figures from the text in PDF and PPTX formats
- ▶ Links to professional-strength computer-aided design (CAD) tools from Intel[®]
- ▶ Instructions on how to use PlatformIO (an extension of Visual Studio Code) to compile, assemble, and simulate C and assembly code for RISC-V processors
- ▶ Hardware description language (HDL) code for the RISC-V processor
- ▶ Intel's Quartus helpful hints
- ▶ Lecture slides in PowerPoint (PPTX) format
- ▶ Sample course and laboratory materials
- ▶ List of errata

The instructor site (accessible to instructors who register at <https://inspectioncopy.elsevier.com>) includes the following:

- ▶ Solutions to all exercises
- ▶ Laboratory solutions

EdX MOOC

This book also has a companion Massive Open Online Course (MOOC) through EdX. The course includes video lectures, interactive practice

该教材的内容适合第一门课程的学生，并展示了书中的微架构如何扩展到现代处理器。

章节末练习和面试问题

学习数字设计的最佳方式是实践。每章都以大量练习来巩固所学内容。练习之后是我们业界同事向申请该领域工作的学生提出的一系列面试问题。这些问题提供了一个有用的视角，让人们了解求职者在面试过程中通常会遇到的问题类型。练习答案可通过本书的配套网站和教师网站获得。

在线补充剂

补充材料可在 ddcabook.com 或出版社网站 <https://www.elsevier.com/books-and-journals/book-companion/9780128200643> 在线获取。这些伴随网站（所有读者均可访问）包括以下内容：

视频讲座链接 ▶ 奇数编号练习题解答 ▶ 教材中的图表 PDF 和 PPTX 格式 ▶ Intel 专业级计算机辅助设计 (CAD) 工具的链接[®] 使用 PlatformIO（Visual Studio Code 的扩展）来编译、汇编和模拟 RISC-V 处理器的 C 和汇编代码的说明 ▶ RISC-V 处理器的硬件描述语言 (HDL) 代码 ▶ Intel Quartus 使用小提示 ▶ PowerPoint (PPTX) 格式的讲座幻灯片 ▶ 课程和实验示例材料 ▶ 勘误表清单

讲师网站（可供在 <https://inspectioncopy.elsevier.com> 注册的讲师访问）包括以下内容：

所有练习的解答 ▶ 实验室溶液

edX 大规模开放在线课程

这本书还有一个通过 EdX 提供的配套大型开放式在线课程（MOOC）。该课程包括视频讲座和互动练习。

problems, and interactive problem sets and labs. The MOOC is divided into two parts: Digital Design (ENGR 85A) and Computer Architecture (ENGR85B) offered by HarveyMuddX (on EdX, search for “Digital Design HarveyMuddX” and “Computer Architecture HarveyMuddX”). EdX does not charge for access to the videos but does charge for the interactive exercises and certificate. EdX offers discounts for students with financial need.

HOW TO USE THE SOFTWARE TOOLS IN A COURSE

Intel’s Quartus Software

The Quartus software, either Web or Lite Edition, is a free version of Intel’s professional-strength Quartus™ FPGA design tools. It allows students to enter their digital designs in schematic or using either the SystemVerilog or the VHDL hardware description language (HDL). After entering the design, students can simulate their circuits using the ModelSim™-Intel FPGA Edition or Starter Edition, which is available with Intel’s Quartus software. Quartus also includes a built-in logic synthesis tool that supports both SystemVerilog and VHDL.

The difference between the Web or Lite Edition and the Pro Edition is that the Web or Lite Edition supports a subset of the most common Altera FPGAs. The free versions of ModelSim degrade performance for simulations with more than 10,000 lines of HDL, whereas the professional version of ModelSim does not.

PlatformIO

PlatformIO, which is an extension of Visual Studio Code, serves as a software development kit (SDK) for RISC-V. With the explosion of SDKs for each new platform, PlatformIO has streamlined the process of programming and using various processors by providing a unified interface for a large number of platforms and devices. It is available as a free download and can be used with SparkFun’s RED-V RedBoard, as described in the labs provided on the companion website. PlatformIO provides access to a commercial RISC-V compiler and allows students to write both C and assembly programs, compile them, and then run and debug them on SparkFun’s RED-V RedBoard (see [Chapter 9](#) and the accompanying labs).

Venus RISC-V Assembly Simulator

The Venus Simulator (available at: <https://www.kvakil.me/venus/>) is a web-based RISC-V assembly simulator. Programs are written (or copy/pasted) in the Editor tab and then simulated and run in the Simulator tab. Registers and memory contents can be viewed as the program runs.

问题，以及交互式问题集和实验。该慕课分为两部分：数字设计（ENGR 85A）和计算机体系结构（ENGR 85B），由HarveyMuddX提供（在EdX上搜索“Digital Design HarveyMuddX”和“Computer Architecture HarveyMuddX”）。EdX不收取观看视频的费用，但会对交互式练习和证书收费。EdX为有经济困难的学生提供折扣。

如何在课程中使用软件工具

英特尔 Quartus 软件

Quartus 软件，无论是 Web 版还是 Lite 版，都是英特尔专业级 Quartus™ FPGA 设计工具的免费版本。它允许学生通过原理图或使用 SystemVerilog 或 VHDL 硬件描述语言 (HDL) 输入他们的数字设计。在输入设计后，学生可以使用 ModelSim™-Intel FPGA 版或 Starter 版模拟他们的电路，这些版本随英特尔的 Quartus 软件提供。Quartus 还包括一个内置的逻辑综合工具，支持 SystemVerilog 和 VHDL。

Web版或简化版与专业版的区别在于，Web版或简化版仅支持最常见的部分Altera FPGA。ModelSim的免费版本在处理超过10,000行HDL的仿真时性能会下降，而ModelSim专业版则不会。

PlatformIO

PlatformIO 是 Visual Studio Code 的一个扩展，作为 RISC-V 的软件开发工具包 (SDK)。随着每个平台 SDK 的激增，PlatformIO 通过为大量平台和设备提供统一界面，简化了编程和使用各种处理器的过程。它可以免费下载，并且可以用于 SparkFun 的 RED-V RedBoard，如配套网站提供的实验中所述。PlatformIO 提供访问商业 RISC-V 编译器的能力，并允许学生编写 C 和汇编程序，编译它们，然后在 SparkFun 的 RED-V RedBoard 上运行和调试（参见第 9 章及附带实验）。

Venus rISc-V 汇编模拟器

金星模拟器（可在：<https://www.kvakil.me/venus/> 获取）是一个基于网络的 RISC-V 汇编模拟器。程序可以在“编辑器”标签中编写（或复制/粘贴），然后在“模拟器”标签中模拟和运行。程序运行时，可以查看寄存器和内存的内容。

LABS

The companion site includes links to a series of labs that cover topics from digital design through computer architecture. The labs teach students how to use the Quartus tools to enter, simulate, synthesize, and implement their designs. The labs also include topics on C and assembly language programming using PlatformIO and SparkFun's RED-V RedBoard.

After synthesis, students can implement their designs using the Altera DE2, DE2-115, DE0, or other FPGA board. The labs are written to target the DE2 or DE-115 boards. These powerful and competitively priced boards are available from de2-115.terasic.com. The board contains an FPGA that can be programmed to implement student designs. We provide labs that describe how to implement a selection of designs on the DE2-115 board using the Quartus software.

To run the labs, students will need to download and install Intel's Quartus Web or Lite Edition and Visual Studio Code with the PlatformIO extension. Instructors may also choose to install the tools on lab machines. The labs include instructions on how to implement the projects on the DE2/DE2-115 board. The implementation step may be skipped, but we have found it of great value. We have tested the labs on Windows, but the tools are also available for Linux.

RVfpga

RISC-V FPGA, also referred to as RVfpga, is a free two-course sequence that can be completed after learning the material in this book. The first course shows how to target a commercial RISC-V core to an FPGA, program it using RISC-V assembly or C, add peripherals to it, and analyze and modify the core and memory system, including adding instructions to the core. This course uses the open-source SweRVolf system-on-chip (SoC) (<https://github.com/chipsalliance/Cores-SweRVolf>), which is based on Western Digital's open-source commercial SweRV EH1 core (<https://www.westerndigital.com/company/innovations/risc-v>). The course also shows how to use Verilator, an open-source HDL simulator, and Western Digital's Whisper, an open-source RISC-V instruction set simulator (ISS). RVfpga-SoC, the second course, shows how to build an SoC based on SweRVolf using building blocks such as the SweRV EH1 core, interconnect, and memories. The course then guides the user in loading and running the Zephyr operating system on the RISC-V SoC. All necessary software and system source code (Verilog/SystemVerilog files) are free, and the courses may be completed in simulation, so no hardware is required. RVfpga materials are freely available with registration from the Imagination Technologies University Programme: <https://university.imgtec.com/rvfpga/>.

实验室

配套网站包括一系列实验的链接，这些实验涵盖从数字设计到计算机体系结构的主题。实验教学生如何使用 Quartus 工具进行设计输入、仿真、综合和实现。实验还包括使用 PlatformIO 和 SparkFun 的 RED-V RedBoard 进行 C 和汇编语言编程的主题。

在综合之后，学生可以使用 Altera DE2、DE2-115、DE0 或其他 FPGA 板来实现他们的设计。这些实验是针对 DE2 或 DE2-115 板编写的。这些功能强大且价格有竞争力的板可以从 de2-115.terasic.com 获得。该板包含一个 FPGA，可以编程来实现学生的设计。我们提供的实验描述了如何使用 Quartus 软件在 DE2-115 板上实现一系列设计。

要运行实验室，学生需要下载并安装英特尔的 Quartus Web 或 Lite 版本以及带有 PlatformIO 扩展的 Visual Studio Code。教师也可以选择实验室计算机上安装这些工具。实验室包括关于如何在 DE2/DE2-115 开发板上实现项目的说明。实现步骤可以跳过，但我们发现它非常有价值。我们已经在 Windows 上测试了这些实验，但这些工具在 Linux 上也可用。

rVfpga

RISC-V FPGA，也称为 RVfpga，是一个免费的两门课程序列，可以在学习本书的内容后完成。第一门课程展示了如何将商业 RISC-V 核心映射到 FPGA 上，使用 RISC-V 汇编或 C 语言进行编程，为其添加外设，并分析和修改核心及内存系统，包括向核心添加指令。该课程使用开源的 SweRVolf 系统芯片（SoC）（<https://github.com/chipsalliance/Cores-SweRVolf>），该芯片基于西部数据的开源商业 SweRV EH1 核心（<https://www.westerndigital.com/company/innovations/risc-v>）。课程还展示了如何使用 Verilator，一款开源 HDL 仿真器，以及西部数据的 Whisper，一款开源 RISC-V 指令集模拟器（ISS）。RVfpga-SoC，即第二门课程，展示了如何使用 SweRV EH1 核心、互连和内存等构建模块基于 SweRVolf 构建 SoC。随后课程将指导

BUGS

As all experienced programmers know, any program of significant complexity undoubtedly contains bugs. So, too, do books. We have taken great care to find and squash the bugs in this book. However, some errors undoubtedly do remain. We will maintain a list of errata on the book's webpage.

Please send your bug reports to ddcabugs@gmail.com. The first person to report a substantive bug with a fix that we use in a future printing will be rewarded with a \$1 bounty!

ACKNOWLEDGMENTS

We appreciate the hard work of Steve Merken, Nate McFadden, Ruby Gammell, Andrae Akeh, Manikandan Chandrasekaran, and the rest of the team at Morgan Kaufmann who made this book happen. We love the art of Duane Bibby, whose cartoons enliven the chapters.

We thank Matthew Watkins, who contributed the section on Heterogeneous Multiprocessors in [Chapter 7](#), and Josh Brake, who contributed to [Chapter 9](#) on Embedded I/O Systems. We greatly appreciate the work of Mateo Markovic and Geordie Ryder, who reviewed the book and contributed to the exercise solutions. Numerous other reviewers also substantially improved the book. They include Daniel Chaver Martinez, Roy Kravitz, Zvonimir Bandic, Giuseppe Di Luna, Steffen Paul, Ravi Mittal, Jennifer Winikus, Hesham Omran, Angel Solis, Reiner Dizon, and Olof Kindgren. We also appreciate the students in our courses at Harvey Mudd College and UNLV who have given us helpful feedback on drafts of this textbook. And last, but not least, we both thank our families for their love and support.

虫子

正如所有有经验的程序员所知，任何具有相当复杂性的程序无疑都会包含漏洞。书籍也是如此。我们已经非常仔细地查找并修正了本书中的错误。然而，某些错误无疑仍然存在。我们将在本书的网页上维护一个勘误表。

请将您的错误报告发送至 ddcbugs@gmail.com。第一个报告一个实质性错误并提供我们在未来印刷中使用的修复的人将获得 1 美元的奖金！

致谢

我们感谢 Steve Merken、Nate McFadden、Ruby Gammell、Andrae Akeh、Manikandan Chandrasekaran 以及 Morgan Kaufmann 其余团队成员的辛勤工作，使这本书得以实现。我们喜爱 Duane Bibby 的艺术，他的漫画使各章生动有趣。

我们感谢 Matthew Watkins，他为第七章的异构多处理器部分做出了贡献；以及 Josh Brake，他为第九章关于嵌入式 I/O 系统的内容做出了贡献。我们非常感谢 Mateo Markovic 和 Geordie Ryder，他们审阅了本书并对习题解答有所贡献。还有许多其他审稿人也大大改进了本书，包括 Daniel Chaver Martinez、Roy Kravitz、Zvonimir Bandic、Giuseppe Di Luna、Steffen Paul、Ravi Mittal、Jennifer Winikus、Hesham Omran、Angel Solis、Reiner Dizon 和 Olof Kindgren。我们还感谢哈维穆德学院和内华达大学拉斯维加斯分校课程中的学生，他们对本教材的草稿提供了有益的反馈。最后但同样重要的是，我们感谢我们的家人给予的爱与支持。

About the Authors

Sarah L. Harris is an Associate Professor of Electrical and Computer Engineering at the University of Nevada, Las Vegas. She received her Ph.D. and M.S. in Electrical Engineering from Stanford University. Sarah has also worked with Hewlett-Packard, the San Diego Supercomputer Center, and NVIDIA. Sarah loves teaching, exploring and developing new technologies, traveling, playing the guitar and various other instruments, and spending time with her family. Her recent research includes designing bio-inspired prosthetics and implementing machine-learning algorithms in hardware.

David Harris is the Harvey S. Mudd Professor of Engineering Design and Associate Department Chair at Harvey Mudd College. He received his Ph.D. in electrical engineering from Stanford University and his M.Eng. in electrical engineering and computer science from MIT. Before attending Stanford, he worked at Intel as a logic and circuit designer on the Itanium and Pentium II processors. Since then, he has consulted at Broadcom, Sun Microsystems, Hewlett-Packard, Evans & Sutherland, and other design companies. David's passions include teaching, building chips and airplanes, and exploring the outdoors. When he is not at work, he can usually be found hiking, flying, and spending time with his family. David holds about a dozen patents and is the author of three other textbooks on chip design, as well as seven guidebooks to the Southern California mountains.

关于作者

莎拉·L·哈里斯是内华达大学拉斯维加斯分校电气与计算机工程的副教授。她在斯坦福大学获得了电气工程的博士和硕士学位。莎拉还曾在惠普公司、圣地亚哥超级计算中心以及英伟达工作。莎拉热爱教学，探索 and 开发新技术，旅行，弹吉他及各种其他乐器，并与家人共度时光。她最近的研究包括设计仿生义肢和在硬件中实现机器学习算法。

大卫·哈里斯是哈佛·穆德学院工程设计哈佛·S·穆德教授兼副系主任。他在斯坦福大学获得电气工程博士学位，并在麻省理工学院获得电气工程和计算机科学的工程硕士学位。在就读斯坦福之前，他曾在英特尔工作，担任Itanium和Pentium II处理器的逻辑与电路设计工程师。从那以后，他曾为博通、太阳微系统、惠普、Evans & Sutherland以及其他设计公司担任顾问。大卫的兴趣包括教学、设计芯片和飞机、以及户外探索。当他不在工作时，通常可以看到他在徒步旅行、飞行或与家人共度时光。大卫拥有大约十二项专利，并且是另外三本芯片设计教科书以及七本南加州山脉指南的作者。

